

Real-Time Voice AI Orchestration

Goal

Build an end-to-end **real-time voice agent** where a user can talk to the agent over **WebRTC** (via **LiveKit**) and the agent can answer using **prompt instructions** and **RAG over uploaded documents** during the call.

Core Requirements

A) Voice Orchestration (LiveKit)

- Use **LiveKit** to enable real-time voice conversation over **WebRTC**.
- Orchestrate the pipeline as **separate components**:
 1. **STT service/component**
 2. **LLM service/component**
 3. **TTS service/component**
 4. **Knowledge Base (KB) ingestion + retrieval component**

B) Knowledge Base + RAG

- Frontend should allow **uploading documents** to create a KB
- Implement ingestion:
 - chunking + embeddings + storage in a vector store (your choice)
- During the **live call**, the agent must:
 - retrieve relevant context from the uploaded KB
 - use it in the LLM response
 - speak the response back via TTS
- In the demo, you must show a query that is clearly answered using the uploaded documents.

C) Frontend (React preferred)

Build a UI with these 3 capabilities:

1. **Talk to the agent over WebRTC** (LiveKit room connect, start/stop, mic controls)
2. **Tweak the agent prompt** (system prompt editable in UI)
3. **Upload documents for KB creation** (basic KB management is fine)

Optional but helpful:

- live transcript view (partial/final)
- “RAG sources used” panel (show retrieved chunks or doc titles)

Deliverables (Submission)

Please submit **all** of the following:

1. **Demo video link** (Google Drive)
 - A short video showing:
 - prompt tweak
 - document upload
 - start call
 - ask a question that triggers RAG from uploaded docs
 - agent answers via voice
 - **No code walkthrough/discussion needed** in the video.
2. **Codebase**
 - Either a **GitHub repo link** (preferred) OR a **zip file**
 - Include a clear **README** with:
 - setup steps
 - environment variables
 - how to run frontend + backend
 - how to run LiveKit (local or cloud)
 - any known limitations/tradeoffs

Evaluation Criteria

We'll review on:

- **End-to-end correctness:** voice call works; RAG works during call
- **Code quality:** separation of concerns, readability, structure
- **Product thinking:** UI clarity, usability, basic error handling
- **Operational maturity (bonus):** Dockerization, simple deployability, logs/metrics hooks